

Commercialising Post-Quantum Cryptography

A practical, plain-English guide to PQC products, enterprise migration,
business models, certification and crypto-agility

Prajjwal Saxena

Companion spreadsheet (business-model scoring, roadmap, unit economics):

PQC Startup Commercialisation Analysis – Google Sheet

<https://docs.google.com/spreadsheets/d/1KcB5NhC5iyS69jfNkiO3VY0aXMeEXOxC1jo6dSmxbCo/edit?usp=sharing>

September 22, 2026

Facts about standards, regulations and deployments were checked against public sources on 21 September 2026. This field moves quickly; always re-check dates and status before relying on them.

Executive Summary

Post-quantum cryptography (PQC) is two problems at once. It is a mathematics problem (which algorithms are safe against quantum computers?) and an enterprise-engineering problem (how do we find, replace, test and certify cryptography across thousands of systems without breaking anything?). The first problem now has official answers. The second is where most of the commercial opportunity lies.

Seven things to remember

1. **The core standards are final.** NIST published FIPS 203 (ML-KEM), FIPS 204 (ML-DSA) and FIPS 205 (SLH-DSA) on 13 August 2024. Two more algorithms, FN-DSA and HQC, are still moving through standardisation.
2. **Hybrid key exchange is already mainstream on the web.** Cloudflare reported around two-thirds of human HTTPS traffic on its network using post-quantum key exchange in April 2026, but only about 9% of origin servers.
3. **Deadlines are now written into policy.** Examples: NIST's draft plan to deprecate today's public-key algorithms after 2030 and disallow them after 2035; the NSA's rule that new national-security purchases follow CNSA 2.0 from 1 January 2027; US Executive Order 14412 (June 2026); and India's 2026 task-force roadmap with milestones from 2027 to 2033.
4. **The hard part is not the algorithm.** It is discovering where vulnerable cryptography lives, replacing it safely, and proving to auditors that it works.
5. **Do not make an unproven algorithm your product.** New schemes have been broken years after publication. Use standardised algorithms as the foundation and add proprietary value above them.
6. **Certification is a long-lead item.** FIPS 140-3 validation has been taking well over a year in the queue alone. Start early.
7. **Sell outcomes, not algorithm names.** Customers buy "we know where our risk is, we are migrating, and we can switch algorithms again later".

The recommended startup path, developed in Part III, is:

Services → Discovery → Migration platform
→ Crypto-agility → Infrastructure and hardware → Optional new research

How this document is organised

Part I – Foundations

The cryptography you need, explained in simple words: the quantum threat, classical algorithms, the NIST PQC algorithms, the enterprise stack (TLS, PKI, HSM, VPN, SSH) and the real costs of PQC.

Part II – The market

Standards, regulation, deadlines and what the market looks like today, including India.

Part III – Building a business

Startup tracks, discovery and CBOM, 18 business models, revenue, certification, randomness, roadmap, risks and a learning path.

Appendices

The companion spreadsheet, a timeline of events, and a full list of abbreviations.

How to read the jargon

Every technical abbreviation is spelled out where it is first used and is also listed in the **Abbreviations** chapter at the end of the document. If a term looks unfamiliar, look it up there first.

Companion spreadsheet

The 18 business models discussed in Chapter 10 were scored, roadmapped and given a unit-economics template in a separate workbook:

PQC Startup Commercialisation Analysis – Google Sheet

The workbook holds the editable numbers (scores, prices, customer counts). This document holds the explanations. Appendix A describes each tab and how to use it. The scores in the workbook are analytical estimates, not market-validated forecasts; replace them with customer-interview data as you collect it.

Contents

Executive Summary	1
I Foundations	6
1 Why PQC Matters	7
1.1 The story in one paragraph	7
1.2 Shor and Grover: what each quantum algorithm attacks	7
1.3 Harvest now, decrypt later and Mosca’s inequality	8
1.4 How big a quantum computer is needed?	8
1.5 Which part is urgent first?	8
1.6 Why this creates a business	8
2 Cryptography Building Blocks	10
2.1 Four things security must provide	10
2.2 Symmetric and asymmetric cryptography	10
2.3 Hash functions	11
2.4 Classical public-key algorithms	11
2.4.1 RSA	11
2.4.2 Diffie–Hellman (DH)	11
2.4.3 Elliptic-curve Diffie–Hellman (ECDH) and X25519	11
2.4.4 ECDSA	11
2.5 What is a KEM?	12
2.6 Random numbers: the hidden ingredient	12
3 The NIST PQC Algorithms	13
3.1 Families of post-quantum maths	13
3.2 The standards and their status	13
3.3 Security categories	14
3.4 Sizes compared with classical algorithms	14
4 The Enterprise Security Stack	16
4.1 TLS: protecting a network connection	16
4.1.1 Hybrid key exchange	17
4.1.2 What is deployed today	17
4.2 PKI and X.509: who do you trust?	17
4.3 HSMs, KMS and PKCS#11: where keys live	18

4.4	OpenSSL and other crypto libraries	18
4.5	VPNs, IPsec and IKEv2	18
4.6	SSH	19
4.7	Where things stand: a quick scorecard	19
5	The Real Cost of PQC: Bandwidth, Memory, CPU and Latency	20
5.1	Bandwidth: a worked example	20
5.2	Authentication is the bigger jump	20
5.3	Memory and code size	21
5.4	CPU, latency and throughput	21
5.5	Implementation security is part of the cost	21
II	The Market	22
6	Standards, Regulation and Deadlines	23
6.1	United States	23
6.2	United Kingdom and European Union	24
6.3	India	24
6.4	Timeline summary	24
7	Market Snapshot and Competition	26
7.1	How big is the market?	26
7.2	Adoption signals	26
7.3	Who competes?	26
7.4	What this means for a startup	27
III	Building a PQC Business	28
8	Three Startup Tracks and the Danger of Inventing Your Own Algorithm	29
8.1	Why a new algorithm is a very different business	29
8.1.1	Attacks a new lattice scheme must survive	30
8.1.2	History is a warning	30
8.2	Fifteen questions to ask any vendor with a home-made algorithm	31
8.3	The recommended architecture	31
9	Discovery, CBOM and Crypto-Agility	33
9.1	The first customer problem: where is our cryptography?	33
9.2	The Cryptographic Bill of Materials (CBOM)	33
9.3	Discovery methods	34
9.4	Prioritising: which systems first?	34
9.5	The migration workflow	35
9.6	Crypto-agility	35
10	The 18 Commercialisation Models	36
10.1	Qualitative comparison	38
11	Revenue Models and Simple Unit Economics	40

11.1 Common revenue models	40
11.2 A few business terms in plain words	40
11.3 An illustrative calculation	41
11.4 Pricing tips for a young company	41
12 Customer Segments	42
13 Trust: Testing, Validation and Certification	43
13.1 What the certificates mean	43
13.2 Validated, compliant or capable?	43
13.3 Time and cost	44
13.4 Strategies for a startup	44
14 Randomness: A Hidden Dependency and an Adjacent Opportunity	45
14.1 How random numbers are produced	45
14.2 Why this matters commercially	45
15 Startup Roadmap and Go-to-Market	47
15.1 Phases	47
15.2 A minimum viable discovery product	48
15.3 Team	48
15.4 Go-to-market	48
16 Risks and Mitigations	50
17 A Learning Path and Hands-On Labs	52
17.1 Suggested order	52
17.2 Labs you can run this week	52
17.3 Exercises	53
17.4 Primary sources worth monitoring	53
18 Final Strategic Recommendation	54
A The Companion Spreadsheet	56
Abbreviations	57
References	62

Part I

Foundations

Chapter 1

Why PQC Matters

1.1 The story in one paragraph

Almost all secure communication on the internet relies on public-key cryptography, mainly RSA, Diffie–Hellman and elliptic-curve schemes. Their security rests on maths problems that ordinary computers cannot solve in reasonable time. In 1994 Peter Shor showed that a large, error-corrected quantum computer could solve exactly these problems efficiently [1]. Such a machine does not exist today. But three facts make the risk urgent anyway:

1. Attackers can **record encrypted traffic now** and decrypt it later.
2. Replacing cryptography across a large organisation takes **many years**.
3. Estimates of how big a quantum computer must be have been **falling**.

1.2 Shor and Grover: what each quantum algorithm attacks

Two quantum algorithms matter.

- **Shor’s algorithm** [1] breaks RSA, Diffie–Hellman (DH), elliptic-curve Diffie–Hellman (ECDH), X25519 and elliptic-curve signatures (ECDSA, EdDSA). This is the serious one.
- **Grover’s algorithm** [2] speeds up brute-force search by a square-root factor. It does not break AES or SHA-2. It effectively halves the “bit strength” of a symmetric key, which is why 256-bit keys are the conservative choice.

Type	Examples	Effect of a large quantum computer
Public-key encryption / key exchange	RSA, DH, ECDH, X25519	Broken by Shor. Must be replaced (for example by ML-KEM).
Public-key signatures	RSA, ECDSA, EdDSA	Broken by Shor. Must be replaced (for example by ML-DSA).
Symmetric encryption	AES-128, AES-256	Grover gives a square-root speed-up. AES-256 keeps a comfortable margin.
Hash functions	SHA-256, SHA-384	Grover and related attacks reduce margins but do not break them.

Table 1.1: What is and is not threatened by quantum computers.

In plain English

Think of public-key cryptography as the *lock on the front door* and AES as the *safe inside the house*. Quantum computers threaten the front-door lock. The safe only needs to be a bit thicker. That is why PQC work focuses on public-key algorithms.

1.3 Harvest now, decrypt later and Mosca's inequality

A “harvest now, decrypt later” (HNDL) attack means recording encrypted data today and waiting for a quantum computer. It matters most for data that must stay secret for many years: health records, defence plans, industrial designs, diplomatic traffic.

Michele Mosca gave a simple planning rule [3]. Let X be how long the data must stay confidential, Y how long migration will take, and Z how long until a cryptographically relevant quantum computer (CRQC) exists. If

$$X + Y > Z,$$

you are already late. For example, if data must stay secret for 15 years and migration will take 8 years, then any CRQC arriving in less than 23 years is a problem. (These numbers are only an illustration.)

1.4 How big a quantum computer is needed?

In 2019 Gidney and Ekâ estimated that factoring a 2048-bit RSA number would take about 20 million noisy physical qubits and roughly eight hours [4]. In May 2025 Gidney published a new analysis showing that fewer than one million noisy qubits could do it in under a week [5]. Further 2026 preprints propose even smaller machines under different assumptions, but these designs are theoretical and have not been built [6]. Nobody knows exactly when a CRQC will exist. The direction of travel, however, is that estimates fall over time, and planners should not assume a comfortable delay.

1.5 Which part is urgent first?

- **Key exchange (how two parties agree on a secret).** Urgent, because of HNDL. Recorded sessions can be attacked later.
- **Signatures (how we prove identity or authenticity).** An attacker must forge a signature at the time it is checked, so there is no “record now” attack. But signature migration is slow because of certificates, root keys, firmware and long-lived documents. The US Executive Order and the NSA schedule therefore give signatures a later, but firm, deadline (Chapter 6).

1.6 Why this creates a business

Regulators have set dates. Migration touches every layer of an organisation. Most organisations do not know where their cryptography is. A 2026 study sponsored by Entrust and run by the Ponemon Institute (more than 4,000 practitioners) reported that 68% of organisations find

managing their cryptographic assets very or extremely difficult, and only 38% said they were actively transitioning to PQC [7]. That gap between deadlines and readiness is the market.

Chapter 2

Cryptography Building Blocks

2.1 Four things security must provide

Confidentiality Outsiders cannot read the data.

Integrity Nobody can change the data without being detected.

Authentication You know who you are talking to (the real bank, not a copy).

Key establishment

Two parties who have never met agree on a shared secret without sending it directly.

2.2 Symmetric and asymmetric cryptography

Symmetric cryptography uses one shared secret key K to lock and unlock:

$$C = \text{Enc}_K(M), \quad M = \text{Dec}_K(C).$$

The Advanced Encryption Standard (AES) is the main example, with 128-, 192- or 256-bit keys. It is very fast and encrypts the bulk of real traffic, usually in a mode called GCM that also checks integrity.

Asymmetric (public-key) cryptography uses a pair of keys: a public key that anyone may see and a private key that stays secret. It is used for two different jobs:

- **Key establishment:** DH, ECDH, X25519, and in the post-quantum world ML-KEM.
- **Digital signatures:** RSA, ECDSA, and in the post-quantum world ML-DSA and SLH-DSA.

People often mix up ECDSA (a signature scheme) and ECDH (a key-agreement scheme). They are different tools.

In plain English

Public-key cryptography is used only briefly, at the start of a connection, to agree on a secret key and to prove identity. After that, fast symmetric encryption (AES) protects the actual data. Replacing the public-key part does *not* mean replacing AES.

2.3 Hash functions

A hash function turns any input into a short, fixed-size fingerprint: $M \mapsto H(M)$. SHA-256 outputs 256 bits and SHA-384 outputs 384 bits (both belong to the SHA-2 family). Good hash functions are one-way (hard to go backwards) and collision-resistant (hard to find two inputs with the same fingerprint). They appear inside signatures, certificates, message authentication (HMAC), key derivation functions (KDFs) and integrity checks.

2.4 Classical public-key algorithms

2.4.1 RSA

RSA is built on the difficulty of factoring a large number $N = pq$ into two primes. The public key contains (N, e) ; the private key involves a secret value d . Typical sizes are RSA-2048, RSA-3072 and RSA-4096. RSA has been used for encryption, key transport and signatures, and is common in older enterprise systems. **Shor's algorithm breaks it.**

2.4.2 Diffie–Hellman (DH)

Alice and Bob want a shared secret over a public channel. They publicly agree on numbers g and p . Alice picks a secret a and sends $A = g^a \bmod p$; Bob picks b and sends $B = g^b \bmod p$. Each can compute the same value $K = g^{ab} \bmod p$, but an eavesdropper who sees only g, p, A, B cannot (with classical computers). **Shor's algorithm breaks it.**

2.4.3 Elliptic-curve Diffie–Hellman (ECDH) and X25519

ECDH does the same job using points on an elliptic curve. With base point G : $A = aG$, $B = bG$ and $K = aB = bA = abG$. It gives shorter keys than DH for the same classical security. X25519 is a specific, widely used ECDH function on Curve25519 [8]. Its public keys are only 32 bytes. **Shor's algorithm breaks it.**

2.4.4 ECDSA

The Elliptic-Curve Digital Signature Algorithm signs a message m with a private key, $\sigma = \text{Sign}(sk, m)$, and anyone can check it with the public key, $\text{Verify}(pk, m, \sigma)$. It provides authentication and integrity, not encryption. **Shor's algorithm breaks it.**

2.5 What is a KEM?

The new NIST key-establishment standards do not work exactly like DH. They are **Key-Encapsulation Mechanisms (KEMs)**, which have three steps:

$$\begin{array}{ll} (pk, sk) \leftarrow \text{KeyGen}() & \text{(receiver makes a key pair)} \\ (ct, K) \leftarrow \text{Encaps}(pk) & \text{(sender makes a secret } K \text{ and a ciphertext } ct) \\ K \leftarrow \text{Decaps}(sk, ct) & \text{(receiver recovers the same } K) \end{array}$$

In DH both parties contribute equally; in a KEM one side “wraps” a fresh random secret for the other. In protocols such as TLS a KEM slots into the same place where ECDH used to sit, so ML-KEM is best described as a *replacement for the key-exchange step*, not a replacement for AES.

2.6 Random numbers: the hidden ingredient

Every algorithm above needs good random numbers for keys, nonces and (for many schemes) every signature or encapsulation. If the randomness is weak, the strongest algorithm fails. Chapter 14 returns to this because it is both a risk and a business opportunity.

Chapter 3

The NIST PQC Algorithms

3.1 Families of post-quantum maths

PQC algorithms are ordinary software and hardware algorithms that run on normal computers. They are “post-quantum” because they are based on maths problems for which no efficient quantum attack is known. The main families are:

Lattice-based	Hard problems about points in high-dimensional grids (Learning With Errors and relatives). Fast and compact enough for general use. ML-KEM, ML-DSA and FN-DSA belong here.
Hash-based	Security relies only on hash functions, which makes these the most conservative choice. SLH-DSA (stateless), and LMS and XMSS (stateful) belong here.
Code-based	Hard problems about decoding error-correcting codes. HQC belongs here, chosen as a backup to lattices.

3.2 The standards and their status

NIST started its PQC competition in 2016. Kyber, the ancestor of ML-KEM, was submitted in 2017 and the first three final standards appeared on 13 August 2024, about eight years after the process began [9, 10]. That history is important for Chapter 8: even a very strong candidate needed years of public attack attempts.

Algorithm	Standard	Job	Maths family	Status (Sept. 2026)
ML-KEM (formerly Kyber)	FIPS 203	Key establish- ment	Module lattice	Final (Aug. 2024) [10]
ML-DSA (formerly Dilithium)	FIPS 204	Signatures	Module lattice	Final (Aug. 2024) [11]

Algorithm	Standard	Job	Maths family	Status (Sept. 2026)
SLH-DSA (formerly SPHINCS+)	FIPS 205	Signatures	Hash-based	Final (Aug. 2024) [12]
FN-DSA (formerly Falcon)	FIPS 206	Compact signatures	NTRU lattice	Still being finalised; not yet a final standard in the sources checked
HQC	expected FIPS 207	Backup key establishment	Code-based	Selected by NIST in March 2025; draft standard pending, final expected around 2027 [13]
LMS, XMSS	SP 800-208	Stateful signatures (firmware)	Hash-based	Approved since 2020 [14]

Table 3.1: NIST post-quantum standards. Status changes; check NIST’s PQC page for the latest.

In plain English

NIST deliberately picked more than one family. If a clever attack ever weakens lattices, hash-based signatures and the code-based HQC would be unaffected. This is called *algorithm diversity*, and it is one reason customers ask for crypto-agility.

3.3 Security categories

NIST describes strength using **categories 1 to 5**, defined by comparison to how hard it is to break AES or SHA-2 variants. Roughly: category 1 is comparable to AES-128 key search, category 3 to AES-192 and category 5 to AES-256. ML-KEM-512, ML-KEM-768 and ML-KEM-1024 target categories 1, 3 and 5; ML-DSA-44, ML-DSA-65 and ML-DSA-87 target categories 2, 3 and 5 [15]. The NSA’s CNSA 2.0 suite asks for the top levels (ML-KEM-1024 and ML-DSA-87) for national-security systems, while many commercial deployments use the middle level (ML-KEM-768, ML-DSA-65).

3.4 Sizes compared with classical algorithms

Post-quantum algorithms are fast but **bigger**. Table 3.2 gives typical sizes in bytes. ML-KEM and ML-DSA numbers come from the NIST standards; X25519, ECDSA and RSA are shown for comparison. FN-DSA figures are approximate because the standard is not final.

Scheme	Public key	Ciphertext / signature	Category
X25519 (classical)	32	32 (key share)	–
ML-KEM-512	800	768	1
ML-KEM-768	1,184	1,088	3
ML-KEM-1024	1,568	1,568	5
ECDSA P-256 (classical)	64	64	–
RSA-2048 (classical)	256	256	–
ML-DSA-44	1,312	2,420	2
ML-DSA-65	1,952	3,309	3
ML-DSA-87	2,592	4,627	5
SLH-DSA-128s (small)	32	7,856	1
SLH-DSA-128f (fast)	32	17,088	1
FN-DSA-512 (approx.)	897	about 666	1

Table 3.2: Approximate sizes in bytes. The shared secret from ML-KEM is 32 bytes.

Chapter 5 shows why these differences matter for real products.

Do not confuse “final standard” with “ready everywhere”

A standard for the algorithm is only step one. Protocols (TLS, IPsec, SSH), certificate formats, libraries, hardware modules and certification schemes all need their own updates. Much of the commercial opportunity lives in that gap.

Chapter 4

The Enterprise Security Stack

Customers do not use “ML-KEM” directly. They use products built from layers. Understanding the layers shows where PQC has to be inserted, and therefore where a startup can sell something.

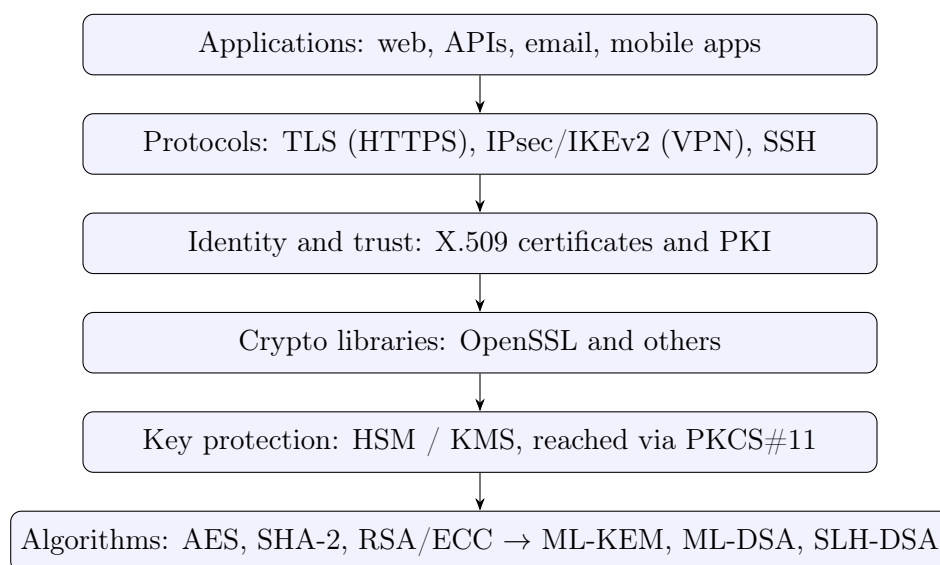


Figure 4.1: The layers of a typical enterprise security stack. PQC touches every layer.

4.1 TLS: protecting a network connection

Transport Layer Security (TLS) protects application traffic such as HTTPS. TLS 1.3 was defined in RFC 8446 and was restated in **RFC 9846** (July 2026), which obsoletes RFC 8446 [28]. A simplified handshake does four things: the two sides agree on settings, the server proves its identity with a certificate, both sides establish a shared secret, and both switch to fast AES encryption.

TLS therefore has two places where PQC is needed:

- **Key exchange** (shared secret): today X25519, tomorrow ML-KEM or a hybrid.
- **Authentication** (certificate and signature): today RSA or ECDSA, tomorrow ML-DSA or similar.

4.1.1 Hybrid key exchange

During the transition most deployments use a **hybrid**: run a classical exchange *and* a post-quantum one, then mix both secrets together. The result stays safe unless both parts are broken. The IETF draft for TLS 1.3 defines groups such as X25519MLKEM768, SecP256r1MLKEM768 and SecP384r1MLKEM1024 [29]. For X25519MLKEM768 the client’s key share is 1,216 bytes (1,184 for ML-KEM plus 32 for X25519) and the server’s share is 1,120 bytes (1,088 plus 32). Hybrid terminology is standardised in RFC 9794 [30].

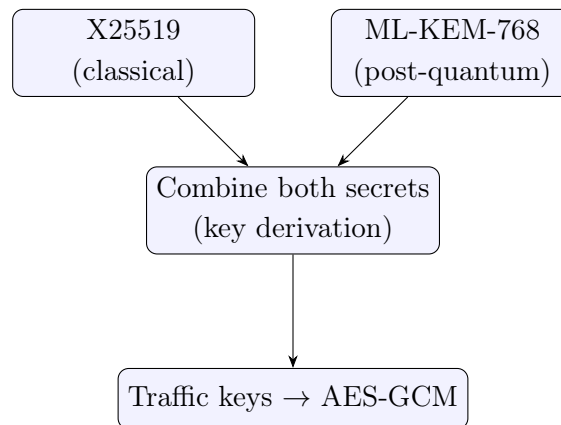


Figure 4.2: Hybrid key exchange: safe unless *both* components are broken.

4.1.2 What is deployed today

Cloudflare’s Radar service reported around 67% of human HTTPS traffic on Cloudflare protected by post-quantum key exchange in April 2026, up from 32% in January 2025, while post-quantum support on origin servers (the customer’s own servers behind Cloudflare) stood near 9% [44]. Cloudflare’s annual review had shown 52% at the end of 2025 [45]. Major browsers offer hybrid key exchange by default [46, 47]. The message for startups: browser-to-edge key exchange is largely solved for large providers; the long tail of enterprise back-ends, internal services and devices is not.

Post-quantum *authentication* on the public web is much less advanced, mainly because certificate chains become large. Cloudflare has begun supporting post-quantum authentication for connections to customer origin servers as a first step [46], and ways to shrink certificate overhead are being explored in the IETF and by browser vendors.

4.2 PKI and X.509: who do you trust?

A Public Key Infrastructure (PKI) is the system that decides which public keys belong to which identities. A Certificate Authority (CA) signs certificates. An X.509 certificate (profiled in RFC 5280 [27]) contains the subject, issuer, public key, validity period, serial number, extensions and the issuer’s signature. Browsers and operating systems trust a set of root CAs; roots sign intermediate CAs; intermediates sign server certificates.

The IETF has published algorithm identifiers so that PQC keys can be placed in certificates:

- RFC 9881 – ML-DSA in X.509 (October 2025) [31];

- RFC 9909 – SLH-DSA in X.509 [32];
- RFC 9935 – ML-KEM in X.509 (March 2026) [33].

Publishing identifiers is only the start. Changing a signature algorithm ripples through root and intermediate CAs, HSMs, issuance software, TLS servers, applications, clients and devices, plus certificate sizes and compatibility testing. That ripple is exactly what a PKI-migration product sells to solve.

4.3 HSMs, KMS and PKCS#11: where keys live

A Hardware Security Module (HSM) is tamper-resistant hardware that stores private keys and performs cryptographic operations so that the key never leaves the device. A bank's CA signing key, for example, should live in an HSM rather than on a normal server. A Key Management Service (KMS) is the software or cloud service that creates, rotates, stores and controls access to keys, often backed by HSMs.

Applications talk to HSMs through standard interfaces. **PKCS#11** (Public-Key Cryptography Standards #11) is the most common. It lets an application ask an HSM to generate a key, sign, verify, encrypt, decrypt or derive a key without knowing that vendor's internals. Version 3.2 of the OASIS PKCS#11 specification adds mechanisms for post-quantum algorithms [36]. HSM firmware, PKCS#11 drivers and validated modules all need PQC updates, which is slow and certification-heavy, and is why HSM and KMS support is a strong but demanding business (Model D in Chapter 10).

4.4 OpenSSL and other crypto libraries

OpenSSL is the most widely used open-source cryptography and TLS toolkit. **OpenSSL 3.5** (April 2025) added ML-KEM, ML-DSA and SLH-DSA in its default provider and makes the hybrid X25519MLKEM768 group a default TLS key-exchange choice [37]. Other important libraries include liboqs (Open Quantum Safe project), AWS-LC, CIRCL for Go, PQClean and pqm4 for ARM Cortex-M4 microcontrollers [39, 40]. For a startup this means two things: basic PQC algorithms are now free, so the value must come from integration, performance, validation, management and support; and integrating with what customers already use (OpenSSL, NGINX, HAProxy, Java, Go) is more realistic than asking them to adopt a proprietary stack.

4.5 VPNs, IPsec and IKEv2

A Virtual Private Network (VPN) creates a protected tunnel across an untrusted network. IPsec protects IP traffic; the Internet Key Exchange (IKEv2) negotiates its keys and the Encapsulating Security Payload (ESP) carries the protected packets. RFC 9370 lets IKEv2 run multiple key exchanges in one setup so that a classical exchange can be combined with one or more post-quantum KEMs [34]. An IETF draft specifies how to use ML-KEM in IKEv2 [35]. A practical difficulty is that IKE messages travel in UDP packets and can be fragmented by larger PQC keys.

4.6 SSH

Secure Shell (SSH) provides encrypted remote access for administrators, developers, cloud and data-centre operations. OpenSSH uses a hybrid key exchange named `mlkem768x25519-sha256`, which became the default in OpenSSH 10.0 [38]. Host and user *authentication* keys, however, are still mostly classical, so an inventory tool must look for both.

4.7 Where things stand: a quick scorecard

Area	Status (Sept. 2026)	Main remaining gap
Web key exchange	Hybrid ML-KEM widely deployed in browsers and CDNs	Origin servers, internal services, older clients
Web authentication	Identifiers published (RFC 9881, 9909, 9935); little public deployment	Certificate size, public-CA issuance, client support
SSH	Hybrid key exchange default in OpenSSH 10	Authentication keys, embedded and legacy systems
IPsec / VPN	Multi-key-exchange framework (RFC 9370); ML-KEM drafts	Vendor implementations, interoperability, fragmentation
HSM / KMS	Vendors adding PQC; PKCS#11 3.2	FIPS 140-3 validation, performance, firmware updates
Code signing	LMS/XMSS approved; ML-DSA available	Ecosystem tooling, long-lived device roots of trust

Table 4.1: Deployment scorecard. “Status” is a summary of public sources, not a survey.

Chapter 5

The Real Cost of PQC: Bandwidth, Memory, CPU and Latency

If PQC is “just new maths”, why is it a hard engineering problem? Because sizes and resource use change.

5.1 Bandwidth: a worked example

In a classical X25519 exchange the client and server each send a 32-byte key share, 64 bytes in total. In hybrid X25519MLKEM768 they send 1,216 and 1,120 bytes, 2,336 bytes in total [29]. The extra data is $2,336 - 64 = 2,272$ bytes per handshake.

- One million handshakes: about **2.3 gigabytes** of extra traffic.
- One billion handshakes: about **2.3 terabytes**.

For one user that is nothing. For telecom networks, satellite links, IoT fleets and huge web services it is a real cost, and packets that no longer fit into one network frame can add extra round trips.

5.2 Authentication is the bigger jump

A back-of-the-envelope comparison: a TLS server typically sends about two public keys (leaf and intermediate certificates) and three signatures (two certificate signatures and one handshake signature). Ignoring other certificate fields:

$$\text{ECDSA P-256: } 2 \times 64 + 3 \times 64 = 320 \text{ bytes,}$$

$$\text{ML-DSA-65: } 2 \times 1,952 + 3 \times 3,309 = 13,831 \text{ bytes.}$$

That is more than 40 times as much, roughly 14 kilobytes. This is why post-quantum *certificates* on the public web are harder than post-quantum *key exchange*, and why compact schemes (FN-DSA), new certificate designs and caching are active topics.

5.3 Memory and code size

On a cloud server with 32 GB of RAM, PQC memory use is irrelevant. On a microcontroller with 256 KB of RAM it can decide whether the product is possible. An embedded PQC implementation needs code space, stack space, buffers for polynomial arrays and temporary variables. Embedded PQC is therefore a distinct engineering niche (projects such as pqm4 target ARM Cortex-M4 chips).

5.4 CPU, latency and throughput

ML-KEM is fast, often comparable to elliptic-curve operations, but hybrids do *both*. ML-DSA verification is quick while signing and signature size matter more. Hash-based signatures such as SLH-DSA have small keys but large signatures and slower signing. For a system that performs a billion handshakes, an extra half millisecond of latency per handshake matters. A serious PQC product should therefore benchmark and publish:

Latency, throughput, CPU load, memory, bandwidth, key size, ciphertext or signature size, energy (for devices)

Who cares most about which cost?

- **HSM buyers:** operations per second.
- **IoT and automotive:** RAM, code size, energy, update size.
- **Telecom and satellite:** bandwidth, round trips, packet size.
- **Web and cloud:** handshake latency, CPU cost at scale, certificate size.

5.5 Implementation security is part of the cost

A correct algorithm can still leak secrets through a poor implementation. For example, timing differences in division operations in some Kyber implementations led to the “KyberSlash” attacks [41]. Production PQC code needs constant-time programming, side-channel and fault-attack resistance, careful input validation and good test vectors. This engineering depth is a genuine competitive moat, because it is hard to copy.

Part II

The Market

Chapter 6

Standards, Regulation and Deadlines

Regulation is the main reason enterprises will buy. This chapter summarises the main frameworks. Some are law or binding policy, some are guidance, and some are drafts. Dates below were checked in September 2026 but can change.

6.1 United States

NIST IR 8547 (draft). The initial public draft (November 2024) proposes that quantum-vulnerable public-key algorithms providing about 112 bits of security (for example RSA-2048 and ECC P-256) be *deprecated* after 2030, and that all quantum-vulnerable public-key algorithms be *disallowed* after 2035 [15]. In the sources checked it had not been finalised. “Deprecated” means do not use in new systems; “disallowed” means not acceptable at all.

CNSA 2.0 (NSA). For national-security systems: ML-KEM-1024, ML-DSA-87, AES-256, SHA-384 or SHA-512, and LMS or XMSS for software and firmware signing. New acquisitions are expected to comply by default from 1 January 2027; networking equipment and software/firmware signing are targeted for exclusive use by 2030; web, cloud and operating systems by 2033; and overall quantum-resistance by 2035 [21]. Although mandatory only for national security systems, it is a widely used reference timeline.

Executive Order 14412 (22 June 2026). “Securing the Nation Against Advanced Cryptographic Attacks” [22, 23]. Highlights from the official text: agencies must name a PQC migration lead within 30 days; within 90 days the Office of Management and Budget (OMB) issues guidance requiring agencies to move high-value assets and high-impact systems to PQC key establishment by **31 December 2030** and PQC digital signatures by **31 December 2031**; within 180 days NIST revises the Cryptographic Module Validation Program (CMVP) processes to accelerate validations and the Federal Acquisition Regulation (FAR) Council proposes a rule requiring covered contractors to comply with PQC-capable FIPS by 31 December 2030; and within 270 days the Cybersecurity and Infrastructure Security Agency (CISA) publishes minimum elements for a cryptographic bill of materials (CBOM).

FIPS 140-2 sunset (21 September 2026). On this date all remaining FIPS 140-2 validation certificates move to the CMVP “Historical” list. Historical modules are not revoked and can remain in existing systems, but new procurements that require validated cryptography increasingly require FIPS 140-3 [19, 20].

6.2 United Kingdom and European Union

The UK National Cyber Security Centre published phased migration guidance in March 2025: complete discovery and an initial plan by 2028, migrate the highest-priority services by 2031, and finish by 2035, recommending ML-KEM-768 and ML-DSA-65 with hybrids encouraged in the transition [24]. The EU’s coordinated implementation roadmap (June 2025) is summarised in the same secondary source as setting roughly 2030 for critical infrastructure and high-risk uses and 2035 for the rest [24]; check the primary documents before making commitments.

6.3 India

India is especially relevant for a startup in this space. In February 2026 the Department of Science and Technology (DST), under the National Quantum Mission, released a task-force report, “Implementation of Quantum Safe Ecosystem in India” [25, 26]. Key points:

- **Testing and certification.** A framework led by the Telecommunication Engineering Centre (TEC) defines assurance levels 1, 2A, 2B, 2C, 3 and 4, from basic conformance up to critical-infrastructure, sovereign-grade assurance, supported by a three-tier national laboratory model.
- **Migration milestones.** Critical Information Infrastructure (CII) sectors such as defence, telecom and power: preparatory work by 2027, migrate high-priority systems by 2028, full adoption by 2029. Other enterprises: 2028, 2030 and 2033 respectively.
- **Vendor obligations.** The plan mandates CBOM submissions from vendors starting in financial year 2027–28, promotes “no new classical-only deployments” at the second milestone, and calls for upgrading PKI, HSMS, key management systems and libraries to PQC-ready versions.
- **Interim measures.** It suggests quantum gateways, quantum VPNs, proxies and tunnels, and quantum random number generators (QRNGs) as stop-gaps.

One think-tank commentary notes that the timelines are ambitious compared with the 2035 target most countries chose, and that the treatment of hybrids of PQC and quantum key distribution (QKD) needs more detail [26]. As far as we know the report is a national roadmap; binding obligations, where they apply, would come through sector regulators and procurement rules, so check the requirements that apply to your customers.

6.4 Timeline summary

When	What	Region
13 Aug 2024	FIPS 203, 204, 205 published	US
Nov 2024	NIST IR 8547 initial public draft	US
Mar 2025	NIST selects HQC	US
Oct 2025	RFC 9881 (ML-DSA in X.509)	IETF
Feb 2026	DST task-force report on quantum-safe ecosystem	India
Mar 2026	RFC 9935 (ML-KEM in X.509)	IETF
22 Jun 2026	Executive Order 14412	US

When	What	Region
Jul 2026	RFC 9846 (TLS 1.3 restated)	IETF
21 Sep 2026	FIPS 140-2 certificates move to Historical	US
1 Jan 2027	New national-security acquisitions expected to follow CNSA 2.0	US
FY 2027–28	Vendor CBOM submissions mandatory (per task-force plan)	India
2027	India CII preparatory milestone; HQC final standard expected	India, US
2028	UK discovery and plan complete; India CII high-priority migration	UK, India
31 Dec 2030	NIST deprecation; EO key-establishment deadline for high-value assets; CNSA 2.0 networking and signing	US
31 Dec 2031	EO deadline for PQC signatures on high-value assets	US
2033	CNSA 2.0 web, cloud and OS; India enterprise full adoption	US, India
2035	NIST disallow date (draft); UK and national-security completion targets	US, UK

Table 6.1: Selected dates. Sources are given in the text above.

In plain English

Different rulebooks use different dates, and some are drafts. A startup should build to the *strictest* rulebook that its target customers follow, and should make it easy for customers to see which rule each of their systems falls under.

Chapter 7

Market Snapshot and Competition

7.1 How big is the market?

Analyst reports disagree widely because each defines “PQC market” differently (software only, or including hardware, services and quantum-key products). Some examples:

- MarketsandMarkets: about USD 0.42 billion in 2025 rising to about USD 2.84 billion by 2030 (about 46% yearly growth) [50].
- Grand View Research: about USD 1.6 billion in 2025 rising to about USD 20.5 billion by 2033 (about 38% yearly growth) [51].

Treat all such numbers as rough. What they agree on is direction: fast growth from a small base, driven by compliance dates.

7.2 Adoption signals

- Cloudflare: about 67% of human HTTPS traffic post-quantum protected in April 2026; origin servers about 9% [44].
- Cloudflare has publicly discussed 2029 as a target for post-quantum security across its products [44].
- Ponemon/Entrust: 68% of organisations find crypto-asset management very or extremely hard; 38% actively transitioning [7].

7.3 Who competes?

Large platform vendors. Cloud providers, network vendors, operating systems and browser makers add PQC to what they already sell. They are a competitor and also a channel.

Cryptography specialists. Companies that license PQC libraries and hardware cores (for example PQShield), or sell discovery and migration platforms (for example SandboxAQ, QuSecure).

PKI and HSM vendors. Established names (for example DigiCert, Keyfactor, Entrust, Thales, Utimaco) extend certificate lifecycle management and HSMs to PQC.

Consultancies and integrators. Large advisory firms and specialist boutiques sell assessments and migration programmes.

Open source. OpenSSL, liboqs, AWS-LC, CIRCL and others make basic algorithms free.

Companies named above are examples drawn from public market reports [51, 50], not an endorsement or a complete list.

7.4 What this means for a startup

1. Basic algorithm code is a commodity. Do not sell it alone.
2. Large vendors will bundle simple PQC support. Compete where they are slow: cross-vendor discovery, migration orchestration, specialised sectors (defence, telecom, embedded), local certification, and performance engineering.
3. Buyers will ask for proof: validated modules, independent testing, interoperability results and a plan for changing algorithms later.
4. Regional advantage exists. National frameworks such as India's task-force plan create demand for locally supported, locally testable products.

Part III

Building a PQC Business

Chapter 8

Three Startup Tracks and the Danger of Inventing Your Own Algorithm

A founder must first decide what to sell. There are three broad tracks.

Track	What you do	Time to a sell-able product	Main risk
A. New primitive	Invent a new key-exchange or signature scheme and license it	2–4 years to deploy with selected customers; 5–10+ years for community confidence	Scientific: one successful attack can invalidate everything
B. Standard algorithms, better implementation	Implement ML-KEM, ML-DSA and others with better performance, hardware, side-channel resistance and certification	About 3–6 months for a first version; 6–18+ months for an enterprise-grade product	Commercial: competition from free and bundled code
C. PQC platform	Build discovery, migration, policy and crypto-agility tools around standard algorithms	About 6–12 months for a first discovery product	Commercial: long sales cycles, big-vendor competition

Table 8.1: The three startup tracks. Time figures are planning estimates, not measured data.

8.1 Why a new algorithm is a very different business

Building a working prototype of a new lattice-based scheme can take months. Trusting it takes years. The people who design a scheme are not the right people to certify it; you need hostile, independent cryptographers trying to break it.

Stage	Typical time	What happens
Mathematical design	6–18 months	Define the scheme, parameters and security argument
Software prototype	3–9 months	Reference code, test vectors
Internal cryptanalysis	1–2 years	Search for attacks and parameter weaknesses
External cryptanalysis	1–3+ years	Independent researchers try to break it
Production implementation	1–2 years	Constant-time code, side-channel and fault resistance, APIs
Protocol integration	6–18 months	TLS, VPN, PKI, HSM, applications
Customer pilots	1–2 years	Real deployments, interoperability
Certification	1–3+ years	FIPS 140-3 and similar, depending on market
Broad acceptance	5–10+ years	Ecosystem confidence and adoption
Formal standardisation	5–10+ years	If entering a NIST or IETF process

Table 8.2: Indicative timeline for a genuinely new scheme (stages overlap). These are planning estimates.

8.1.1 Attacks a new lattice scheme must survive

Plain-English versions of the checks designers face:

- **Primal, dual and hybrid lattice attacks and BKZ estimates** – the main ways to attack the hard lattice problem; used to estimate the true security level.
- **Decryption-failure and reaction attacks** – exploiting the rare cases where a scheme decrypts wrongly, or how a server reacts to bad input.
- **Algebraic and combinatorial attacks** – shortcuts using the mathematical structure.
- **Quantum attacks** – any speed-up that a quantum computer gives.
- **Implementation attacks** – timing, power, electromagnetic and fault attacks on real code and hardware.

8.1.2 History is a warning

Even well-studied candidates have failed late. SIKE, a NIST round-4 candidate, was broken in 2022 by a classical attack that recovered keys on an ordinary computer in a short time [42]. Rainbow, a NIST signature finalist, was broken the same year using a laptop over a weekend [43]. ML-KEM itself took about eight years from submission to final standard, and has still needed careful implementation work (for example the KyberSlash timing issues [41]).

Risk of tying your product to a proprietary algorithm

A software company can patch a bug. A cryptography company whose only product is a new algorithm can lose its whole foundation to a single paper. This is why the safe

pattern is

Customer security = ML-KEM+Experimental scheme not Experimental scheme alone.

Combine the experimental scheme with a standard one so customers are never guinea pigs.

8.2 Fifteen questions to ask any vendor with a home-made algorithm

1. Is the algorithm actually new, or a variant of a published one?
2. Where is the security proof or reduction?
3. Who has independently attacked it, and where are the results?
4. What security level is claimed (which NIST category)?
5. Against which hard problem is it claimed to be secure?
6. What are the exact parameters?
7. What are the public-key and ciphertext (or signature) sizes?
8. What is the decryption-failure probability?
9. Has it been analysed against BKZ, primal, dual and hybrid attacks?
10. Is there constant-time production code?
11. Has side-channel analysis been done?
12. Which standards does it implement or comply with?
13. Does it work with existing TLS and PKI infrastructure?
14. Does the customer need FIPS 140-3 validation, and can it be achieved?
15. What happens commercially if the algorithm is broken?

The last question matters most.

8.3 The recommended architecture

In plain English

Use **standardised PQC underneath** and put **proprietary value above**: discovery, automation, performance, hardware, integration, management, support and certification. Keep new algorithms in a research track until public analysis has earned trust.

Sanity check: what if a startup says “we will ship our own algorithm in 12 months”?

A working prototype in six months is plausible. A trustworthy, certified, interoperable product built on a brand-new primitive in twelve months is not. Ask for independent cryptanalysis reports before reading anything else.

Chapter 9

Discovery, CBOM and Crypto-Agility

9.1 The first customer problem: where is our cryptography?

The first question every organisation faces is: *where exactly are we using vulnerable public-key cryptography?* It may hide in servers, source code, binaries, cloud services, TLS endpoints, certificates, VPNs, SSH, HSMs, firmware, embedded devices and third-party software. The NIST National Cybersecurity Center of Excellence (NCCoE) migration project names discovery and inventory as core work streams, alongside risk prioritisation, interoperability and migration [17].

9.2 The Cryptographic Bill of Materials (CBOM)

A CBOM is to cryptography what a software bill of materials (SBOM) is to software dependencies: a machine-readable inventory of algorithms, keys, certificates, protocols and the libraries that implement them. CBOM support was added to the CycloneDX standard in version 1.6 (developed with IBM Research) and CycloneDX is now published as Ecma standard ECMA-424 [48]. Open-source tools such as IBM's CBOMkit can already generate CBOMs from code and container images [49]. CBOMs are moving from good practice to requirement: India's plan mandates vendor CBOMs from FY 2027–28, and US Executive Order 14412 directs CISA to publish minimum CBOM elements within 270 days [26, 22].

A simplified inventory looks like this:

Asset	Algorithm	Purpose	Risk context	Candidate action
API server	RSA-2048	TLS identity	Internet-facing	Plan PQC certificate path
VPN gateway	ECDH X25519	/ Key establish-ment	Remote access	Test PQC or hybrid path
Firmware	ECDSA	Code signing	Long-lived devices	Plan PQC or hash-based signature
Database	AES-256	Data encryption	Stored data	Symmetric layer can remain; review keys

Table 9.1: A tiny example inventory (CBOM view).

The inventory should record the algorithm and also its purpose, owner, asset criticality and how long the protected data must stay confidential.

```
# Simplified illustration of one CBOM entry (CycloneDX-style JSON).
# Field names are indicative; see the CycloneDX specification for exact schema.
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "components": [{
    "type": "cryptographic-asset",
    "name": "RSA-2048",
    "cryptoProperties": {
      "assetType": "algorithm",
      "algorithmProperties": {
        "primitive": "signature",
        "parameterSetIdentifier": "2048",
        "nistQuantumSecurityLevel": 0
      }
    }
  ]
}
```

9.3 Discovery methods

Method	What it finds	Limitation
Network scanning	TLS versions, cipher suites, certificate chains, SSH algorithms	Sees only reachable services
Certificate inventory	Issuers, key types, expiry, owners	Misses non-certificate crypto
Source-code scanning	Library calls, hard-coded algorithms	False positives; build-time detail
Binary and container scanning	Statically linked libraries and versions	Slower; needs symbols
Configuration and cloud APIs	KMS keys, storage encryption, cloud TLS settings	Vendor-specific connectors
Runtime observation	Algorithms actually used in production	Agents and privacy concerns
Hardware and firmware review	HSMs, secure elements, device roots of trust	Manual effort; vendor cooperation

Table 9.2: Common discovery methods and their limits.

9.4 Prioritising: which systems first?

A simple planning heuristic combines four factors:

$$\text{Priority} \propto \text{Exposure} \times \text{Data lifetime} \times \text{Asset criticality} \times \text{Migration difficulty}.$$

This is a startup planning heuristic, not a NIST formula. The point is that internet-facing, long-lived, high-value and hard-to-change systems deserve attention first, and that long-lived data is at HNDL risk sooner.

9.5 The migration workflow

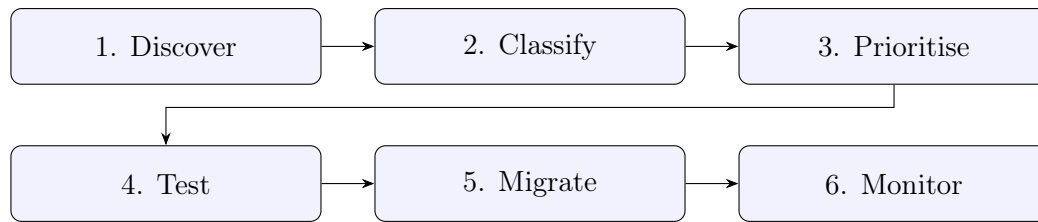


Figure 9.1: Enterprise PQC migration workflow. “Monitor” feeds back into “Discover”.

1. **Discover** public-key cryptography in applications, networks, certificates, devices and HSMs.
2. **Classify**: algorithm, key size, purpose, owner, asset, data lifetime.
3. **Prioritise** long-lived and high-value information.
4. **Test** in a non-production PQC or hybrid environment.
5. **Migrate** using standard profiles, in stages, with rollback plans.
6. **Monitor** algorithms, certificates, dependencies and policy continuously.

9.6 Crypto-agility

Crypto-agility means designing systems so algorithms can be changed without rebuilding the whole application. NIST has published CSWP 39 on strategies and practices for achieving it [16].

Rigid design:	Application -> ECDH -> AES	(algorithm hard-coded everywhere)
Agile design:	Application -> Crypto policy layer -> { X25519 ML-KEM future KEM }	

If ECDH is hard-coded in 10,000 applications, replacing it means 10,000 rewrites. If applications call a policy layer, the change is made once. A crypto-agility product typically provides:

- algorithm selection by policy,
- inventory and version management,
- migration workflows and testing,
- monitoring, exceptions and rollback,
- reporting for auditors.

```

# Illustrative policy (not a real product format)
policy:
  tls_key_exchange:  [X25519MLKEM768, SecP384r1MLKEM1024]
  tls_signatures:    [ECDSA-P256]          # move to ML-DSA when ecosystem is ready
  code_signing:      [LMS, ML-DSA-87]
  deprecate_after:   { RSA-2048: 2030-12-31 }
  
```

Chapter 10

The 18 Commercialisation Models

This chapter describes 18 ways to build a PQC business. The spreadsheet [52] contains the numeric scoring for each model across market need, recurring revenue, scalability, differentiation, enterprise value, execution feasibility, time-to-revenue and technical moat, plus an overall attractiveness score:

[PQC Startup Commercialisation Analysis – Google Sheet](#)

Each model below is described the same way: what it is, who pays, how it earns, where the advantage lies, and the main risk.

Software and services layer

Model A – PQC cryptographic library

What: optimised, hardened implementations of ML-KEM, ML-DSA, SLH-DSA and related algorithms. **Who pays:** software and device makers. **Revenue:** licences, support and update subscriptions. **Edge:** speed, constant-time and side-channel resistance, small footprint, service-level agreements (SLAs). **Risk:** free open-source libraries reduce willingness to pay.

Model B – PQC SDK or API

What: a developer-friendly layer that hides algorithm choice, for example `SecureChannel(policy="PQC-HYBRID")`. **Revenue:** per device, per server, per API call or annual enterprise licence. **Edge:** developer experience and crypto-agility built in. **Risk:** platform vendors bundling similar features.

Model C – PQC-as-a-Service

What: cloud operations such as key generation, signing, verification and key encapsulation. **Revenue:** usage-based. **Risk:** customers with very sensitive keys prefer on-premises or HSM-backed designs.

Model F – PQC migration platform

What: discover, inventory, classify, prioritise, test, migrate and monitor (Chapter 9). **Revenue:** SaaS per asset, application or enterprise. **Edge:** cross-vendor visibility and automation. **Risk:**

long sales cycles; large vendors expanding downward. This model closely matches the needs described by NIST NCCoE [17].

Model P – PQC compliance platform

What: inventory-backed evidence, policies, exceptions and audit reports. **Edge:** strongest when connected to real technical discovery, not just a reporting dashboard. **Risk:** becoming a checklist tool without technical depth.

Model K – Managed PQC service

What: assessment, implementation and ongoing monitoring delivered as a service. **Revenue:** assessment fee plus implementation plus annual service. **Why it is attractive early:** it earns revenue before a full platform exists.

Model L – PQC consulting

What: readiness assessments and migration roadmaps. **Advice:** treat consulting as a way to learn customer problems and to productise repeated work later; it does not scale by itself.

Network and infrastructure layer

Model G – PQC gateway

What: a gateway that terminates classical connections and establishes PQC or hybrid connections toward the outside world. **Value:** migrate security properties without rewriting every legacy application (India's task-force report also mentions gateways, VPNs and proxies as interim measures). **Risk:** the gateway becomes a single trust point and must itself be certified.

Model H – PQC VPN

What: a complete VPN product with PQC or hybrid key establishment. **Revenue:** per user, per device or subscription. **Risk:** a crowded market with strong incumbents; differentiation must be clear (for example defence-grade certification).

Model I – PQC PKI and certificate management

What: certificate issuance, rotation, migration, device identity and inventory. **Revenue:** per certificate, per device or subscription. **Edge:** deep PKI migration know-how; certificate hierarchies are hard to change.

Model D – PQC HSM or KMS

What: PQC-capable HSM or KMS infrastructure, sold or leased. **Revenue:** hardware plus software plus maintenance plus support. **Barriers:** high technical and certification barriers, which is also the moat.

Hardware, IP and OEM layer

Model E – PQC accelerator

What: field-programmable gate array (FPGA), application-specific integrated circuit (ASIC) or SmartNIC acceleration for PQC operations. **Value:** higher throughput, lower latency and lower CPU load. **Risk:** long development cycles and capital needs.

Model M – OEM PQC licensing

What: license an implementation or SDK to vendors that already sell VPNs, firewalls, HSMs, PKI or networking equipment. **Revenue:** licence fees and royalties. **Risk:** depends on a few large customers; long negotiations.

Model O – Patent and implementation IP

What: patent implementation or hardware techniques and license them to chip, HSM, cloud or networking companies. **Risk:** patents in a field built on open standards can be hard to enforce or design around.

Model N – Proprietary PQC algorithm licensing

What: develop a new KEM or signature scheme and license it. This is the *highest-risk* model (Chapter 8): a working implementation does not prove security, and trust needs extensive public analysis. Treat it as research, not as the main product.

Vertical markets

Model J – Secure firmware signing

What: PQC signatures plus key management, secure boot and over-the-air (OTA) update security for automotive, aerospace, defence, telecom and industrial devices. **Why now:** devices deployed today may still be in service when quantum computers arrive, and the NSA schedule treats software and firmware signing as an early priority.

Model Q – Telecom-specific PQC

What: 5G and 6G infrastructure, edge systems, network management, device identity and long-lived telecom equipment. **Edge:** deep knowledge of telecom standards. **Risk:** long procurement and standards-driven cycles.

Model R – Automotive and embedded PQC

What: embedded SDK, secure boot, firmware signing, OTA security, device identity and hardware acceleration. **Constraint:** tiny memory, long product lifetimes and strict safety processes.

10.1 Qualitative comparison

Table 10.1 is a quick, qualitative summary written for this document. The detailed numeric scoring is in the spreadsheet; where the two differ, prefer the spreadsheet after you update it

with your own customer research.

Model	Time to revenue	Capital need	Recurring revenue	Moat potential
A Library	Medium	Low–Med	Medium	Low–Med
B SDK/API	Medium	Low–Med	High	Medium
C PQC-as-a-Service	Medium	Medium	High	Low–Med
D HSM/KMS	Slow	High	Medium	High
E Accelerator	Slow	High	Low–Med	High
F Migration platform	Medium	Medium	High	Medium–High
G Gateway	Medium	Medium	Medium	Medium
H VPN	Medium	Medium	High	Low–Med
I PKI/certificate mgmt	Medium	Medium	High	Medium
J Firmware signing	Medium–Slow	Medium	Medium	Medium–High
K Managed service	Fast	Low	Medium	Low
L Consulting	Fast	Low	Low	Low
M OEM licensing	Slow–Med	Low–Med	Medium	Medium
N Proprietary algorithm	Very slow	Med–High	Low	Uncertain
O Patent/IP	Slow	Medium	Low–Med	Medium
P Compliance platform	Medium	Low–Med	High	Low–Med
Q Telecom	Slow	Med–High	Medium	Medium–High
R Automotive/embedded	Slow	Medium–High	Medium	Medium–High

Table 10.1: Qualitative comparison of the 18 models (author’s judgement, for discussion).

In plain English

The pattern: fast-to-start models (services, consulting) fund learning; recurring software models (discovery, migration, PKI) build a business; slow, capital-heavy models (HSM, ASIC, telecom, automotive) build a moat.

Chapter 11

Revenue Models and Simple Unit Economics

11.1 Common revenue models

- **Subscription:** $\text{Revenue} = \text{Customers} \times \text{Annual subscription}$. Fits discovery, compliance and crypto-agility platforms.
- **Per-device licence:** $\text{Revenue} = \text{Devices} \times \text{Licence per device}$. Fits automotive and embedded systems.
- **Usage-based:** $\text{Revenue} = \text{Usage} \times \text{Price}$. Fits cloud APIs.
- **Project plus recurring:** $\text{Revenue} = \text{Implementation} + \text{Annual subscription} + \text{Support}$. Fits enterprise migration.
- **Hardware plus software plus maintenance:** fits HSMs and gateways.
- **Licence and royalty:** fits OEM and IP models.

11.2 A few business terms in plain words

ARR	Annual recurring revenue: subscription revenue counted on a yearly basis.
ACV	Annual contract value: what one customer pays per year.
CAC	Customer acquisition cost: sales and marketing cost to win one customer.
Gross margin	Share of revenue left after direct costs of delivering it.
Payback period	Months needed to recover CAC from a customer's gross profit.

11.3 An illustrative calculation

The numbers below are *hypothetical*, not forecasts, and are not taken from the spreadsheet. Replace them with your own in the Unit Economics tab.

$$\begin{aligned} \text{ACV} &= \text{USD } 60,000 \text{ per customer,} & \text{Customers} &= 20, \\ \text{ARR} &= 20 \times 60,000 = \text{USD } 1,200,000, \\ \text{Gross margin} &= 75\% \Rightarrow \text{gross profit per customer} = 45,000, \\ \text{CAC} &= \text{USD } 40,000, \\ \text{Payback} &= \frac{40,000}{45,000} \times 12 \approx 10.7 \text{ months.} \end{aligned}$$

Enterprise security sales are often slow, so model the sales cycle, pilot duration and procurement delays explicitly; they usually matter more than the list price.

11.4 Pricing tips for a young company

- Start with a paid pilot or assessment; free proofs of concept teach little.
- Price discovery by assets, applications or business units so it grows with the customer.
- For hardware-linked products, budget separately for certification cost and time.
- Offer one clear package for compliance-driven buyers (for example “CNSA 2.0 readiness”) because they buy against a checklist.

Chapter 12

Customer Segments

Customer	Main problem	Likely products
Banks and payments	TLS, PKI, HSMs, long-lived data, strict audit	Migration SaaS, PKI, HSM/KMS
Government	Long-lived sensitive information; new mandates	Discovery, managed migration, PKI
Defence	Secure communications, embedded and legacy systems	VPN, HSM, embedded SDK, gateways
Telecom	Huge networks and long-lived equipment	Gateway, accelerator, PKI, Model Q
Cloud and internet	Massive TLS and certificate infrastructure	Libraries, gateways, agility
Automotive	Long-lived electronic control units (ECUs) and OTA updates	Embedded SDK, signing, secure boot
IoT and industrial	Limited memory and compute; decades of service	Lightweight implementation, accelerators
Security vendors	Need PQC inside their own products	OEM SDK and licensing
Large enterprises	Unknown crypto inventory	Discovery and migration SaaS
Small and medium enterprises	Limited in-house expertise	Managed PQC service

Table 12.1: Customer segments and the products that fit them.

In plain English

Choose *one* segment first. A defence buyer, a bank and a car maker need different certifications, sales methods and product shapes. Depth in one segment beats breadth in all.

Chapter 13

Trust: Testing, Validation and Certification

Buyers rarely accept “trust us”. Three things must not be confused:

Algorithm security $\neq$ Implementation security $\neq$ Certification.

13.1 What the certificates mean

- **Algorithm validation (CAVP/ACVP).** The Cryptographic Algorithm Validation Program checks that an implementation produces correct outputs for a NIST-approved algorithm, using the Automated Cryptographic Validation Protocol (ACVP).
- **Module validation (FIPS 140-3, run by CMVP).** Accredited laboratories test a whole cryptographic *module* (hardware, software or firmware) against FIPS 140-3, which is based on ISO/IEC 19790 and includes requirements on non-invasive (side-channel) attacks [19]. It concerns the implementation, not whether a mathematical paper looks convincing.
- **Entropy validation (ESV).** Modules that rely on a noise source need the entropy source assessed too (see Chapter 14).
- **Other schemes.** Common Criteria and the NSA’s National Information Assurance Partnership (NIAP) protection profiles for defence products, and, in India, the TEC-led assurance levels 1 to 4 described in the DST task-force report [26].

13.2 Validated, compliant or capable?

Vendors use three phrases with very different meanings:

FIPS validated A specific module version holds a current certificate.

FIPS compliant The product uses approved algorithms but has no certificate for that module (a claim, not a certificate).

FIPS capable The product could be configured to use approved algorithms.

Always ask which one is meant and check the certificate in the public CMVP database.

13.3 Time and cost

FIPS 140-3 validation has been slow. Data cited by one certification vendor showed the CMVP's own queue portion averaging 542 days for FIPS 140-3 versus 367 days for FIPS 140-2, before counting laboratory testing and preparation [20]. Executive Order 14412 directs NIST to revise CMVP processes to accelerate validations, which may improve this [22]. Plan for at least a year of lead time and treat any promise of a fast certificate with caution.

13.4 Strategies for a startup

1. Where possible, build on an already validated module for the cryptographic core and validate only your integration.
2. Design the architecture so the security boundary (the part that must be validated) is small.
3. Keep validated and experimental code strictly separate.
4. Run your own conformance tests early, using public test vectors, and publish interoperability results.
5. Engage a testing laboratory early; requirements interpretation is a real source of delay.

Chapter 14

Randomness: A Hidden Dependency and an Adjacent Opportunity

Post-quantum algorithms need good random numbers just like classical ones. ML-KEM key generation and encapsulation consume fresh random bytes. ML-DSA signing, in its default “hedged” variant, also mixes in randomness. If the random source is predictable, an attacker may recover keys without ever breaking the algorithm.

14.1 How random numbers are produced

A typical system uses a small amount of *true* unpredictability from a physical noise source (an entropy source), then stretches it with a deterministic random bit generator (DRBG). NIST’s SP 800-90 series covers this: SP 800-90A (DRBG mechanisms), SP 800-90B (entropy sources) and SP 800-90C (how to combine them into random bit generators) [18]. In FIPS 140-3 validation, modules that depend on an entropy source need it assessed through the Entropy Source Validation (ESV) process.

14.2 Why this matters commercially

- HSMS, secure elements, embedded chips and PQC accelerators all need validated entropy sources; a weak or unvalidated source can block certification.
- Hardware random number generators can be sold as IP, chips or modules alongside PQC products, and some vendors already bundle quantum random number generators (QRNGs) with PQC offerings (Chapter 6 noted that India’s task force lists QRNGs among interim measures).
- Health monitoring, start-up tests and continuous testing of the noise source are part of a credible product.

Do not confuse the two ideas

A quantum random number generator is a *source of randomness*. It does not make an algorithm quantum-resistant. Post-quantum security comes from the algorithms (ML-

KEM, ML-DSA and so on); randomness quality is a separate, necessary ingredient. Claims about any entropy source should be backed by statistical tests and an entropy assessment, not by the physics story alone.

Chapter 15

Startup Roadmap and Go-to-Market

15.1 Phases

The staged plan below follows the logic of the earlier chapters: earn revenue and learning first, then build recurring software, then infrastructure and hardware, and keep new research optional. Time ranges overlap on purpose.

When	Build	Proof of success	Notes
0–6 months (services-led entry)	PQC readiness assessment; TLS and certificate discovery; basic cryptographic inventory; risk report; migration roadmap	Paid assessments; 3–5 design partners; repeatable report template	Learn real customer problems; avoid building a platform before you know them
6–12 months (discovery SaaS)	Agents and network scanning; cloud connectors; CBOM export; dependency graph; risk dashboard	First recurring contracts; CBOM accepted by customers and auditors	Focus on one segment
12–24 months (migration platform)	ML-KEM and ML-DSA support; hybrid TLS; certificate migration; OpenSSL integration; performance testing	Production pilots; documented interoperability results	Build on validated or well-reviewed cryptographic cores
18–36 months (crypto-agility)	Policy engine; algorithm abstraction; switching; continuous monitoring; migration automation	Customers switching algorithms with no rewrite	Now the platform is hard to replace
24–48 months (infrastructure)	Gateway; SDK; HSM/KMS integration; PKCS#11; hardware acceleration	Validated modules; OEM partnerships	Certification cost and time dominate

When	Build	Proof of success	Notes
3–8+ years (research option)	New KEMs and signatures; implementation IP; FPGA/ASIC; patents; standards work	Independent crypt-hybrid analysis; deployments only	Must never carry the core business

Table 15.1: Recommended startup roadmap.

15.2 A minimum viable discovery product

A concrete first product that can be built in a few months:

1. Scan a list of internet-facing hosts and record TLS version, key-exchange group, certificate key type, signature algorithm and expiry.
2. Scan SSH servers for key-exchange and host-key algorithms.
3. Ingest a customer’s certificate store or CA inventory.
4. Optionally scan a code repository or container image for cryptographic library use.
5. Export a CBOM and a prioritised report (Chapter 9) with plain-language recommendations.

This is deliberately simple. Its value is trust, learning and data, not technical novelty.

15.3 Team

- Applied cryptographer (algorithms, security levels, hybrids).
- Secure-implementation engineer (constant-time C or Rust, side channels).
- Protocol engineer (TLS, IPsec/IKEv2, SSH, X.509).
- PKI and HSM engineer (CA operations, PKCS#11).
- Product and sales lead with enterprise-security experience.
- Compliance and certification lead (FIPS 140-3, Common Criteria, local schemes).

15.4 Go-to-market

- **Design partners first.** Two or three lighthouse customers in one segment shape the product and provide references.
- **Publish evidence.** Benchmarks, interoperability tables and open test results build credibility faster than marketing.
- **Use channels.** Cloud marketplaces, system integrators and OEM partners shorten sales cycles.

- **Know the buyer's rulebook.** A defence buyer asks about CNSA 2.0 and national assurance schemes; a bank asks about audit and HSMS; a telecom operator asks about standards and scale.
- **India.** Follow the DST task-force framework, the TEC testing and certification levels and sector-regulator requirements; early alignment can turn local certification into an advantage [26].

Chapter 16

Risks and Mitigations

Risk	Why it matters	Mitigation
Implementation error	A correct algorithm with a buggy implementation is not secure	Independent code review, test vectors, fuzzing, formal methods where feasible
Side-channel leakage	Timing, power or fault attacks recover keys (for example Kyber-Slash)	Constant-time coding, hardware countermeasures, side-channel testing
Poor parameters or protocol mistakes	Mixing the wrong algorithm or hybrid design breaks security	Follow published standards and profiles; do not invent combiners
Interoperability failure	Products from different vendors fail to connect	Interop testing events, public test suites, conservative defaults
Performance problems	Larger keys and signatures hurt latency, memory or bandwidth	Benchmark per segment; offer hybrid and compact options
Cryptanalytic surprise	New attacks can weaken a scheme (SIKE, Rainbow)	Crypto-agility; algorithm diversity (hybrids, HQC, hash-based)
Standards change	Drafts move; identifiers and profiles change	Track IETF and NIST work; avoid shipping unstable drafts as fixed features
Slow procurement	Enterprise and government sales cycles are long	Services revenue, paid pilots, channel partners
Big-vendor competition	Cloud, network and PKI vendors bundle PQC	Cross-vendor value, sector focus, speed, local certification
Open-source commoditisation	Free libraries reduce the price of algorithms	Sell integration, validation, management and support
Certification delay	FIPS 140-3 and national schemes take a long time	Early lab engagement, small security boundary, validated cores
Regulatory ambiguity	Draft dates and regional variations	Design to the strictest applicable rulebook; keep evidence configurable

Risk	Why it matters	Mitigation
Proprietary algorithm failure	One attack can destroy the product	Keep as R&D; always pair with a standard algorithm
Overclaiming	Selling “quantum-proof” without evidence damages trust	Make precise, sourced claims; publish limits and assumptions

Table 16.1: Main risks and mitigations.

Chapter 17

A Learning Path and Hands-On Labs

17.1 Suggested order

Use this order rather than learning topics at random.

Level 1 – Primitives

Symmetric versus asymmetric cryptography; AES; SHA-256 and SHA-384; RSA; DH; ECDH; X25519; ECDSA.

Level 2 – Infrastructure

TLS; X.509 and PKI; OpenSSL.

Level 3 – Enterprise security

HSMs; PKCS#11; SSH; IPsec and VPNs.

Level 4 – PQC migration

ML-KEM; ML-DSA; SLH-DSA; hybrids; crypto-agility; CBOM and discovery; performance (bandwidth, memory, latency, CPU).

The path in one line:

Mathematics → Cryptography → Protocols → Infrastructure → PQC → Commercialisation.

For the startup objective in this document, the most valuable chain is TLS → X.509/PKI → OpenSSL → HSM/PKCS#11 → VPN/IPsec/SSH → crypto-agility → ML-KEM/ML-DSA integration.

17.2 Labs you can run this week

The commands below use OpenSSL 3.5 or later and OpenSSH 10 or later. Names and options can vary by version and distribution; run `openssl list -help` and `ssh -Q kex` to check what your build supports.

```
# 1. Check the version and the post-quantum algorithms available
openssl version
openssl list -kem-algorithms      | grep -i ml
openssl list -signature-algorithms | grep -i ml
```

```
# 2. Generate an ML-DSA-65 key pair
openssl genpkey -algorithm ML-DSA-65 -out mldsa65.pem

# 3. Connect to a server and request a hybrid key exchange group
#     (look for the negotiated group in the output)
openssl s_client -connect example.com:443 -groups X25519MLKEM768

# 4. See which SSH key-exchange methods your client supports
ssh -Q kex | grep -i mlkem
```

17.3 Exercises

1. Measure handshake sizes with and without the hybrid group using a packet capture tool and compare with the numbers in Chapter 5.
2. Scan ten public websites for key-exchange groups and certificate types and build a tiny CBOM.
3. Read one algorithm standard (FIPS 203 is a good start) and list the byte sizes it defines.
4. Take a small service you know and write a one-page migration plan using the workflow in Chapter 9.

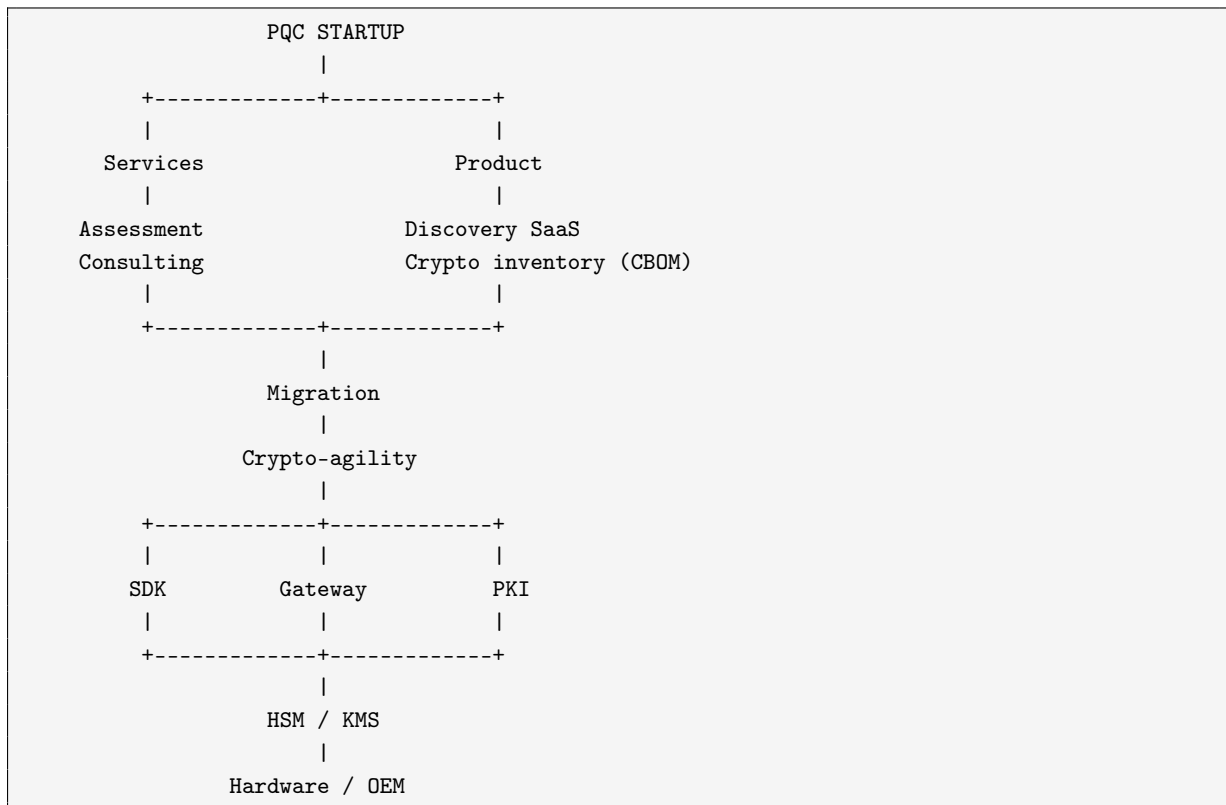
17.4 Primary sources worth monitoring

- NIST PQC project page and NIST Special Publications on transition [9, 15].
- The IETF TLS, LAMPS, IPSECME and PQUIP working-group pages [29, 31, 33, 34, 30].
- The NIST CMVP database for certificate status [19].
- CISA, NSA and, for India, DST and TEC announcements [21, 25].

Chapter 18

Final Strategic Recommendation

The most commercially coherent, general-purpose PQC startup architecture is:



Ten-point summary

1. The main PQC standards (FIPS 203, 204, 205) are final; use them.
2. Key exchange is the first migration target; signatures follow more slowly.
3. Hybrid deployments are the practical bridge.
4. The commercial problem is discovery, migration, integration and proof.
5. Do not build a business on an unproven algorithm.
6. Use standardised PQC underneath and proprietary value above.
7. Start with services, learn, then productise (discovery → migration → crypto-agility).
8. Benchmark bandwidth, memory, CPU and latency for each segment.

9. Start certification early; it is a long-lead item.
10. Build to the strictest rulebook your customers follow, and stay crypto-agile because rules and algorithms will change again.

Do not sell only “PQC”. Sell discovery, migration, deployment and long-term crypto-agility.

A new cryptographic primitive can remain a research option. The production business can rest on standardised algorithms while the moat is built around integration, automation, performance, hardware, PKI, HSMs and enterprise operations.

Appendix A

The Companion Spreadsheet

PQC Startup Commercialisation Analysis – Google Sheet

<https://docs.google.com/spreadsheets/d/1KcB5NhC5iyS69jfNkiO3VY0aXMeEXOxC1jo6dSmxbCo/edit?usp=sharing>

The workbook contains the following tabs.

Tab	Purpose
Executive Summary	Overall commercial conclusions and the recommended startup strategy
PQC Business Models	The 18 models with missing fields completed (what, who pays, revenue, risk)
Commercial Analysis	Scores for each model on market need, recurring revenue, scalability, differentiation, enterprise value, execution feasibility, time-to-revenue and technical moat, plus overall attractiveness
Recommended Roadmap	The phased plan from the first six months to eight or more years (summarised in Chapter 15)
Unit Economics	Editable pricing, customer counts, revenue, CAC, margin and recurring-revenue model (see Chapter 11 for the method)
Scoring Notes	Definitions and assumptions behind the scores

How to use the scores

The scores are analytical estimates. They are not market-validated forecasts. Replace them with evidence from customer interviews, pilots and pricing tests, and update Table 10.1 in this document if your scores change.

Abbreviations

Cryptography and security terms are written out in full where they first appear. This table lists them all in one place with a plain-English meaning.

Abbreviation	Full form	Simple meaning
ACV	Annual Contract Value	What one customer pays per year.
ACVP	Automated Cryptographic Validation Protocol	The automated way NIST tests that an algorithm implementation gives correct results.
AES	Advanced Encryption Standard	The main symmetric (same-key) encryption algorithm; key sizes 128, 192 or 256 bits.
API	Application Programming Interface	A way for one piece of software to call another.
ARR	Annual Recurring Revenue	Subscription income counted per year.
ASIC	Application-Specific Integrated Circuit	A chip built for one job, for example fast PQC.
BKZ	Block Korkine–Zolotarev (algorithm)	A standard lattice-reduction method used to estimate how hard lattice problems are.
CA	Certificate Authority	An organisation that issues and signs digital certificates.
CAC	Customer Acquisition Cost	Sales and marketing cost to win one customer.
CAVP	Cryptographic Algorithm Validation Program	NIST programme that validates algorithm implementations.
CBOM	Cryptographic Bill of Materials	A machine-readable inventory of all cryptography a system uses.
CDN	Content Delivery Network	Global network of servers that delivers websites quickly.
CII	Critical Information Infrastructure	Systems vital to a nation, such as defence, telecom and power.
CISA	Cybersecurity and Infrastructure Security Agency	US agency for cyber and infrastructure defence.
CMVP	Cryptographic Module Validation Program	NIST and Canadian programme that certifies cryptographic modules (FIPS 140-3).
CNSA 2.0	Commercial National Security Algorithm Suite 2.0	The NSA’s list of quantum-resistant algorithms and deadlines for national-security systems.
CPU	Central Processing Unit	The main processor of a computer.
CRQC	Cryptographically Relevant Quantum Computer	A quantum computer big and reliable enough to break today’s public-key cryptography.

Abbreviation	Full form	Simple meaning
CSWP	Cybersecurity White Paper (NIST)	A NIST discussion document, such as CSWP 39 on crypto-agility.
DH	Diffie–Hellman	Classical method for two parties to agree on a shared secret.
DRBG	Deterministic Random Bit Generator	Software that stretches a small random seed into many random bits.
DST	Department of Science and Technology (India)	Indian ministry department that released the quantum-safe task-force report.
ECC	Elliptic-Curve Cryptography	Public-key cryptography built on elliptic curves.
ECDH	Elliptic-Curve Diffie–Hellman	Diffie–Hellman using elliptic curves.
ECDSA	Elliptic-Curve Digital Signature Algorithm	Classical elliptic-curve signature scheme.
ECU	Electronic Control Unit	A small computer inside a vehicle.
EdDSA	Edwards-curve Digital Signature Algorithm	A modern elliptic-curve signature family (for example Ed25519).
EO	Executive Order	A directive from the US President, for example EO 14412.
ESP	Encapsulating Security Payload	The IPsec part that encrypts and protects packets.
ESV	Entropy Source Validation	Testing that a noise source really provides enough randomness.
EU	European Union	–
FAR	Federal Acquisition Regulation	Rules for US government purchasing.
FIPS	Federal Information Processing Standards	US government standards, for example FIPS 203.
FN-DSA	FFT over NTRU-Lattice-Based Digital Signature Algorithm	NIST signature scheme based on Falcon; standard still being finalised.
FPGA	Field-Programmable Gate Array	Reprogrammable chip often used to prototype fast hardware.
FY	Financial Year	The accounting year (India uses April to March).
GCM	Galois/Counter Mode	A mode of AES that encrypts and checks integrity at once.
HMAC	Hash-based Message Authentication Code	A keyed hash that checks a message was not changed.
HNDL	Harvest Now, Decrypt Later	Recording encrypted data today to decrypt it once quantum computers exist.
HQC	Hamming Quasi-Cyclic	Code-based key-establishment scheme selected by NIST as a backup to ML-KEM.
HSM	Hardware Security Module	Tamper-resistant device that stores keys and does cryptography.
HTTPS	Hypertext Transfer Protocol Secure	Web traffic protected by TLS.
IETF	Internet Engineering Task Force	Body that publishes internet standards (RFCs).
IKE, IKEv2	Internet Key Exchange (version 2)	The IPsec protocol that agrees on keys.

Abbreviation	Full form	Simple meaning
IoT	Internet of Things	Small connected devices such as sensors and meters.
IP	Internet Protocol	The basic addressing and routing layer of the internet.
IPsec	Internet Protocol Security	A suite that protects IP traffic, often used in VPNs.
IPSECME, LAMPS, PQUIP	IETF working groups	Groups working on IPsec, certificate and PQC-transition standards.
IR	Internal Report (NIST)	A NIST report series, for example IR 8547.
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission	International standards bodies; ISO/IEC 19790 underlies FIPS 140-3.
JSON	JavaScript Object Notation	A common text format for structured data.
KDF	Key Derivation Function	Turns a shared secret into one or more usable keys.
KEM	Key-Encapsulation Mechanism	Method where one party “wraps” a fresh secret for another using a public key.
KMS	Key Management Service	System that creates, stores, rotates and controls keys.
LMS	Leighton–Micali Signature	A stateful hash-based signature scheme approved in NIST SP 800-208.
ML-DSA	Module-Lattice-Based Digital Signature Algorithm	NIST signature standard (FIPS 204), formerly Dilithium.
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism	NIST key-establishment standard (FIPS 203), formerly Kyber.
NCCoE	National Cybersecurity Center of Excellence	NIST centre running the PQC migration project.
NIAP	National Information Assurance Partnership	US scheme evaluating security products for national security use.
NIST	National Institute of Standards and Technology	US standards agency behind the PQC standards.
NSA	National Security Agency	US agency setting rules for national-security systems.
NTRU	N-th degree Truncated polynomial Ring Units	A family of lattice problems used by FN-DSA.
OASIS	Organization for the Advancement of Structured Information Standards	Body that maintains PKCS#11.
OEM	Original Equipment Manufacturer	A company that builds another company’s technology into its own products.
OMB	Office of Management and Budget (US)	White House office that issues guidance to agencies.
OS	Operating System	Software such as Linux, Windows or Android.
OTA	Over-the-Air (update)	Software or firmware updates sent wirelessly.
PKCS#11	Public-Key Cryptography Standards #11	Standard interface for talking to HSMs and tokens.

Abbreviation	Full form	Simple meaning
PKI	Public Key Infrastructure	System for identities, certificates, keys and trust.
PQC	Post-Quantum Cryptography	Cryptography designed to resist quantum and classical attacks.
QKD	Quantum Key Distribution	Sharing keys using quantum physics over special links; a different approach from PQC.
QRNG	Quantum Random Number Generator	A random number source based on quantum effects.
R&D	Research and Development	–
RAM	Random-Access Memory	A computer’s working memory.
RFC	Request for Comments	Numbered internet standards documents, for example RFC 9846.
RSA	Rivest–Shamir–Adleman	Classical public-key system based on factoring.
SaaS	Software as a Service	Software sold as a subscription over the internet.
SBOM	Software Bill of Materials	An inventory of the components inside a piece of software.
SDK	Software Development Kit	Tools and libraries for developers.
SHA-2 (SHA-256, SHA-384, SHA-512)	Secure Hash Algorithm 2	Family of hash functions; the number is the output size in bits.
SIKE	Supersingular Isogeny Key Encapsulation	A NIST candidate broken by a classical attack in 2022.
SLA	Service-Level Agreement	A contract promise about service quality and response time.
SLH-DSA	Stateless Hash-Based Digital Signature Algorithm	NIST hash-based signature standard (FIPS 205), formerly SPHINCS+.
SP	Special Publication (NIST)	A NIST guidance series, for example SP 800-208.
SSH	Secure Shell	Protocol for secure remote login and administration.
TEC	Telecommunication Engineering Centre (India)	Indian body leading PQC testing and certification frameworks.
TLS	Transport Layer Security	Protocol that protects web and application traffic (used by HTTPS).
UDP	User Datagram Protocol	A simple network transport; IKE messages travel over it.
UK, US	United Kingdom, United States	–
USD	United States Dollar	Currency used in examples.
VPN	Virtual Private Network	An encrypted tunnel across an untrusted network.
X.509	X.509 certificate standard	The common format for digital certificates.
X25519	X25519 elliptic-curve function	A very widely used elliptic-curve key-agreement function.
X25519-MLKEM768	X25519 combined with ML-KEM-768	The hybrid key-exchange group used in TLS today.

Abbreviation	Full form	Simple meaning
XMSS	eXtended Merkle Signature Scheme	A stateful hash-based signature scheme approved in NIST SP 800-208.

Software and project names used in the text: liboqs (Open Quantum Safe library), AWS-LC (Amazon’s cryptography library), CIRCL (Cloudflare’s Go cryptography library), PQClean (portable C implementations), pqm4 (PQC for ARM Cortex-M4 chips), CycloneDX (bill-of-materials standard, Ecma ECMA-424), OpenSSL and OpenSSH (open-source security software).

Bibliography

- [1] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” *Proc. 35th IEEE Symposium on Foundations of Computer Science (FOCS)*, 1994, pp. 124–134.
- [2] L. K. Grover, “A fast quantum mechanical algorithm for database search,” *Proc. 28th ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [3] M. Mosca, “Cybersecurity in an era with quantum computers: will we be ready?” *IEEE Security & Privacy*, vol. 16, no. 5, 2018.
- [4] C. Gidney and M. Ekå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” *Quantum*, vol. 5, art. 433, 2021. <https://doi.org/10.22331/q-2021-04-15-433>
- [5] C. Gidney, “How to factor 2048 bit RSA integers with less than a million noisy qubits,” arXiv:2505.15917, 2025. <https://arxiv.org/abs/2505.15917>
- [6] The Quantum Insider, “Why 2026 matters for quantum security,” 28 April 2026. <https://thequantuminsider.com/2026/04/28/why-2026-matters-quantum-security/>
- [7] Encryption Consulting, “PQC migration in 2026: building a roadmap that survives contact with production” (summarising the 2026 Ponemon Institute study sponsored by Entrust), August 2026. <https://www.encryptionconsulting.com/pqc-migration-in-2026/>
- [8] A. Langley, M. Hamburg and S. Turner, *Elliptic Curves for Security*, RFC 7748, IETF, 2016. <https://www.rfc-editor.org/info/rfc7748>
- [9] NIST, *Post-Quantum Cryptography* project. <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [10] NIST, *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*, 2024. <https://csrc.nist.gov/pubs/fips/203/final>
- [11] NIST, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, 2024. <https://csrc.nist.gov/pubs/fips/204/final>
- [12] NIST, *FIPS 205: Stateless Hash-Based Digital Signature Standard*, 2024. <https://csrc.nist.gov/pubs/fips/205/final>
- [13] NIST, *IR 8545: Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process*, 2025. <https://csrc.nist.gov/pubs/ir/8545/final>
- [14] NIST, *SP 800-208: Recommendation for Stateful Hash-Based Signature Schemes*, 2020. <https://csrc.nist.gov/pubs/sp/800/208/final>
- [15] NIST, *IR 8547 (initial public draft): Transition to Post-Quantum Cryptography Standards*, November 2024. <https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8547.ipd.pdf>
- [16] NIST, *CSWP 39: Considerations for Achieving Cryptographic Agility: Strategies and Practices*, 2025. <https://csrc.nist.gov/pubs/cswp/39/final>

- [17] NIST NCCoE, *Migration to Post-Quantum Cryptography* project. <https://www.nccoe.nist.gov/applied-cryptography/migration-to-pqc>
- [18] NIST, *Random Bit Generation* project page (SP 800-90 series: 90A, 90B, 90C). <https://csrc.nist.gov/projects/random-bit-generation>
- [19] NIST, *FIPS 140-3 Transition Effort*. <https://csrc.nist.gov/projects/fips-140-3-transition-effort>
- [20] SafeLogic, “What happens on September 21, 2026?” (CMVP validation-time data), 2025. <https://www.safelogic.com/blog/what-happens-on-september-21-2026>
- [21] NSA, *Commercial National Security Algorithm Suite 2.0* advisory and FAQ; dates summarised in Encryption Consulting, “What is CNSA 2.0?”, 2026. <https://www.encryptionconsulting.com/education-center/what-is-cnsa-2-0/>
- [22] The White House, *Executive Order 14412: Securing the Nation Against Advanced Cryptographic Attacks*, 22 June 2026. <https://www.whitehouse.gov/presidential-actions/2026/06/securing-the-nation-against-advanced-cryptographic-attacks/>
- [23] Federal Register, document 2026-12909 (Executive Order 14412), 25 June 2026. <https://www.federalregister.gov/d/2026-12909>
- [24] Encryption Consulting, “Post-quantum cryptography migration deadlines by country: complete 2026 global guide,” June 2026. <https://www.encryptionconsulting.com/pqc-migration-deadlines-by-country/>
- [25] Department of Science and Technology (India), Task Force Report, *Implementation of Quantum Safe Ecosystem in India*, February 2026. https://dst.gov.in/sites/default/files/Report_TaskForce_PQMigraion_4Feb26%20%28v1%29.pdf
- [26] P. Tripathi, “India’s post-quantum cryptography migration roadmap,” Observer Research Foundation, 23 March 2026. <https://www.orfonline.org/expert-speak/india-s-post-quantum-cryptography-migration-roadmap>
- [27] D. Cooper *et al.*, *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, RFC 5280, IETF, 2008. <https://www.rfc-editor.org/rfc/rfc5280>
- [28] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 9846, IETF, 2026 (obsoletes RFC 8446). <https://www.rfc-editor.org/info/rfc9846>
- [29] K. Kwiatkowski, P. Kampanakis, B. E. Westerbaan and D. Stebila, *Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3*, Internet-Draft draft-ietf-tls-ecdhe-mlkem (work in progress; version 05 of May 2026). <https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/>
- [30] *Terminology for Post-Quantum Traditional Hybrid Schemes*, RFC 9794, IETF. <https://www.rfc-editor.org/info/rfc9794>
- [31] J. Massimo, P. Kampanakis, S. Turner and B. E. Westerbaan, *Internet X.509 Public Key Infrastructure – Algorithm Identifiers for the Module-Lattice-Based Digital Signature Algorithm (ML-DSA)*, RFC 9881, IETF, October 2025. <https://www.rfc-editor.org/info/rfc9881>
- [32] K. Bashiri *et al.*, *Internet X.509 Public Key Infrastructure – Algorithm Identifiers for the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA)*, RFC 9909, IETF. <https://www.rfc-editor.org/rfc/rfc9909.html>
- [33] S. Turner, P. Kampanakis, J. Massimo and B. E. Westerbaan, *Internet X.509 Public Key Infrastructure – Algorithm Identifiers for the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM)*, RFC 9935, IETF, March 2026. <https://www.rfc-editor.org/info/rfc9935>

- [34] C. Tjhai *et al.*, *Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2)*, RFC 9370, IETF, May 2023. <https://www.rfc-editor.org/info/rfc9370>
- [35] P. Kampanakis and G. Ravago, *Post-quantum key exchange with ML-KEM in the Internet Key Exchange Protocol Version 2 (IKEv2)*, Internet-Draft draft-ietf-ipsecme-ikev2-mlkem (work in progress). <https://datatracker.ietf.org/doc/draft-ietf-ipsecme-ikev2-mlkem/>
- [36] OASIS PKCS 11 Technical Committee, *PKCS #11 Specification, Version 3.2*. <https://www.oasis-open.org/committees/pkcs11/>
- [37] OpenSSL Project, “OpenSSL 3.5 final release,” 8 April 2025. <https://openssl-library.org/post/2025-04-08-openssl-35-final-release/>
- [38] OpenSSH project, *Release notes* (OpenSSH 10.0 and later). <https://www.openssh.com/releases.html>
- [39] Open Quantum Safe project (liboqs). <https://openquantumsafe.org>
- [40] *Study of post quantum status of widely used protocols*, arXiv:2603.28728, 2026. <https://arxiv.org/abs/2603.28728>
- [41] D. J. Bernstein *et al.*, *KyberSlash: exploiting secret-dependent division timings in Kyber implementations*. <https://kyberslash.cr.yp.to/>
- [42] W. Castryck and T. Decru, “An efficient key recovery attack on SIDH,” IACR ePrint 2022/975 (EUROCRYPT 2023). <https://eprint.iacr.org/2022/975>
- [43] W. Beullens, “Breaking Rainbow takes a weekend on a laptop,” IACR ePrint 2022/214 (CRYPTO 2022). <https://eprint.iacr.org/2022/214>
- [44] Cloudflare Radar, post-quantum encryption share of human HTTPS traffic (15 April 2026) and Radar post-quantum page. <https://radar.cloudflare.com/post-quantum>
- [45] Cloudflare, *The 2025 Cloudflare Radar Year in Review*. <https://blog.cloudflare.com/radar-2025-year-in-review/>
- [46] Cloudflare, “State of the post-quantum Internet in 2025.” <https://blog.cloudflare.com/pq-2025/>
- [47] PostQuantumSecurity.org, “From X25519 to X25519+MLKEM768: how hybrid TLS is becoming real,” March 2026. <https://www.postquantumsecurity.org/publications/X25519+MLKEM768.html>
- [48] OWASP CycloneDX (ECMA-424), *Cryptography Bill of Materials (CBOM)*. <https://cyclonedx.org/capabilities/cbom/>
- [49] IBM Research, *CBOM and CBOMkit*. <https://github.com/IBM/CBOM>
- [50] MarketsandMarkets, post-quantum cryptography market forecast, as reported in PR Newswire, 19 August 2026. <https://www.prnewswire.com/news-releases/global-post-quantum-cryptography-market-size-projected-to-reach-2-84-billion-by-2030--302855225.html>
- [51] Grand View Research, *Post-Quantum Cryptography Market Size Report, 2026–2033*. <https://www.grandviewresearch.com/industry-analysis/post-quantum-cryptography-market-report>
- [52] *PQC Startup Commercialisation Analysis* (Google Sheet). <https://docs.google.com/spreadsheets/d/1KcB5NhC5iyS69jfNkiO3VY0aXMeEXOxC1jo6dSmxbCo/edit?usp=sharing>